

Università di Messina — CdL in Informatica — Reti e Sistemi Distribuiti Mod.B

Hands-On #15

Produttore/Consumatore: da zero alle condition variables

Last Update: 6 maggio 2026

Scenario e struttura dell'HO

In questo Hands-On costruirete un programma produttore/consumatore **in quattro step progressivi**. Ogni step aggiunge un pezzo alla step precedente; ogni step introduce deliberatamente un problema che la step successiva risolve. L'obiettivo non è solo scrivere il codice corretto, ma capire *perché* ogni primitiva è necessaria osservando cosa succede senza di essa.

step	File	Cosa aggiunge	Problema osservabile
1	step1.c	Buffer + operazioni, un solo thread	nessuno — baseline corretta
2	step2.c	Due thread, nessuna sincronizzazione	corruzione dei dati
3	step3.c	Aggiunta del mutex	deadlock quando il buffer è vuoto/pieno
4	step4.c	Aggiunta delle condition variables	soluzione corretta e completa

Compilazione comune a tutte le fasi:

```
gcc -Wall -Wextra -pthread -o stepN stepN.c
```

Il buffer condiviso

Tutte e quattro le fasi usano la stessa struttura dati: un buffer circolare di **N** interi con un indice di testa (**head**), uno di coda (**tail**) e un contatore (**count**).

Listing 1: Struttura condivisa da tutte le fasi

```
1  #define N 8      /* capacita' del buffer */
2
3  typedef struct {
4      int buf[N];
5      int head, tail, count;
6  } Buffer;
7
8  /* Inserisce v nel buffer. Non controlla se e' pieno. */
9  void buf_put(Buffer *b, int v) {
10     b->buf[b->tail] = v;
11     b->tail = (b->tail + 1) % N;
12     b->count++;
13 }
14
15 /* Preleva un elemento dal buffer. Non controlla se e' vuoto. */
16 int buf_get(Buffer *b) {
17     int v = b->buf[b->head];
18     b->head = (b->head + 1) % N;
19     b->count--;
20     return v;
21 }
```

▷ Suggerimento

`buf_put` e `buf_get` non fanno nessun controllo intenzionalmente: la responsabilità di non chiamarle in condizioni non valide spetta al chiamante. Nelle prime fasi questo causerà problemi visibili.

step 1 — Buffer a singolo thread

Obiettivo

Verificare che il buffer funzioni correttamente in assenza di concorrenza. Questa step è la **baseline**: se non funziona qui, il problema è nel buffer, non nella sincronizzazione.

Cosa fare

Il file `step1.c` è già completo. Compilatelo ed eseguitelo. Il programma inserisce i numeri da 0 a 15 nel buffer (a gruppi di `N=8`) e li preleva, stampando ogni valore estratto.

1. Verificate che l'output mostri i numeri nell'ordine corretto (FIFO: 0, 1, 2, ..., 15).
2. Modificate `N` a 4 e ripetete: l'ordine deve restare corretto.
3. Cosa succede se chiamate `buf_get` su un buffer vuoto? Provatelo aggiungendo una chiamata in più e osservate l'output.

Output atteso

```
Prelevo: 0
Prelevo: 1
Prelevo: 2
...
Prelevo: 15
Tutti gli elementi prelevati correttamente.
```

step 2 — Due thread senza sincronizzazione

Obiettivo

Introdurre la concorrenza *senza* nessuna protezione e osservare la corruzione dei dati. Non è un esercizio da correggere: è un esercizio da **osservare e capire**.

Il programma

Il produttore inserisce i numeri da 0 a `NITEM-1` nel buffer uno alla volta. Il consumatore li preleva e li stampa. Entrambi girano in loop. Nessun mutex, nessuna CV.

Listing 2: Produttore e consumatore senza sincronizzazione

```
1  #define NITEM 32
2
3  void *produttore(void *arg) {
4      Buffer *b = (Buffer *)arg;
5      for (int i = 0; i < NITEM; i++) {
6          /* Aspetta attivamente che ci sia spazio */
7          while (b->count == N)
8              ;
9          buf_put(b, i);
10     }
11     return NULL;
12 }
```

```

13
14 void *consumatore(void *arg) {
15     Buffer *b = (Buffer *)arg;
16     for (int i = 0; i < NITEM; i++) {
17         /* Aspetta attivamente che ci sia qualcosa */
18         while (b->count == 0)
19             ;
20         printf("Prelevo: %d\n", buf_get(b));
21     }
22     return NULL;
23 }

```

Cosa fare

1. Compilate ed eseguite `step2.c` **più volte**. L'output cambia tra un'esecuzione e l'altra?
2. Eseguite con `NITEM = 1000` e cercate valori duplicati o mancanti nell'output.
3. Eseguite con `valgrind -tool=helgrind ./step2` e leggete i warning. Cosa segnala?
4. Rispondete: perché il programma *sembra* funzionare a volte, anche senza sincronizzazione?

[!] Attenzione

Non tutti i bug da race condition si manifestano ad ogni esecuzione. Un programma che “funziona” senza sincronizzazione funziona *per caso*, non per correttezza. Helgrind lo vede sempre, anche quando l'output sembra corretto.

Cosa sta succedendo

Le istruzioni `b->count++` e `b->count-` non sono atomiche: si traducono in tre istruzioni macchina (leggi, modifica, scrivi). Se produttore e consumatore eseguono queste sequenze in modo interlacciato, il valore finale di `count` può essere sbagliato. Esempio con `count = 1`:

Produttore	Consumatore
legge <code>count = 1</code>	
	legge <code>count = 1</code>
scrive <code>count = 2</code>	
	scrive <code>count = 0</code>

Il risultato corretto sarebbe `count = 1` (una insert, una remove). Il risultato osservato è `count = 0`: una delle due operazioni è andata persa.

step 3 — Aggiunta del mutex

Obiettivo

Proteggere il buffer con un mutex ed osservare che la corruzione scompare, ma compare un nuovo problema: il **deadlock**.

Cosa fare

Partite da `step2.c` e aggiungete un `pthread_mutex_t` alla struttura condivisa. Modificate produttore e consumatore in modo che ogni accesso al buffer sia protetto dal mutex.

1. Compilate ed eseguite. La corruzione è scomparsa?
2. Eseguite con `NITEM = 1000`. Il programma termina sempre?
3. Provate a riprodurre il deadlock: cosa succede se il produttore trova il buffer pieno mentre tiene il mutex?
4. Rispondete: perché tenere il mutex dentro il loop di attesa rende il deadlock inevitabile?

Il codice con mutex

Listing 3: Produttore con mutex — deadlock quando il buffer è pieno

```
1 void *produttore(void *arg) {
2     Shared *s = (Shared *)arg;
3     for (int i = 0; i < NITEM; i++) {
4         pthread_mutex_lock(&s->mtx);
5
6         while (s->b.count == N) /* buffer pieno: aspetta */
7             ;                  /* ma il mutex e' tenuto! */
8
9         buf_put(&s->b, i);
10        pthread_mutex_unlock(&s->mtx);
11    }
12    return NULL;
13 }
```

Perché va in deadlock

Quando il buffer è pieno, il produttore entra nel loop di attesa tenendo il mutex. Il consumatore, per prelevare un elemento e liberare spazio, deve acquisire lo stesso mutex — ma non può, perché il produttore lo tiene. Nessuno dei due avanza.

▷ La soluzione parziale sbagliata

Si potrebbe pensare di rilasciare il mutex dentro il loop:

```
1 while (s->b.count == N) {
2     pthread_mutex_unlock(&s->mtx);
3     /* <-- finestra: il consumatore puo' segnalare qui */
4     pthread_mutex_lock(&s->mtx);
5 }
```

Questo elimina il deadlock, ma introduce una **finestra di race**: se il consumatore modifica `count` e segnalasse qui, il segnale andrebbe perso e il produttore potrebbe aspettare per sempre. Il problema non è risolvibile a livello utente: serve che il rilascio del mutex e la sospensione avvengano in modo atomico. È esattamente ciò che fa `pthread_cond_wait`.

step 4 — Soluzione con condition variables

Obiettivo

Aggiungere le condition variables per eliminare sia il deadlock che la finestra di race, ottenendo la soluzione corretta e completa.

La struttura condivisa finale

Listing 4: Struttura condivisa con mutex e due CV

```

1  typedef struct {
2      Buffer      b;
3      pthread_mutex_t  mtx;
4      pthread_cond_t  not_full;  /* segnalata quando si libera uno slot */
5      pthread_cond_t  not_empty; /* segnalata quando arriva un elemento */
6  } Shared;

```

Due CV distinte per le due direzioni di blocco: `not_full` per il produttore che aspetta spazio, `not_empty` per il consumatore che aspetta elementi. Usarne una sola con `broadcast` funzionerebbe, ma sveglierebbe inutilmente anche i thread che non possono procedere.

Cosa fare

Partite da `step3.c` e completate `step4.c` sostituendo i loop di busy-wait con le chiamate corrette a `pthread_cond_wait` e `pthread_cond_signal`.

1. Completate il produttore: quando il buffer è pieno, invece di girare a vuoto, chiamate `pthread_cond_wait` sulla CV corretta.
2. Completate il consumatore in modo speculare.
3. Identificate dove chiamare `pthread_cond_signal` in ciascuno dei due thread e giustificate la scelta: perché `signal` e non `broadcast`?
4. Eseguite con `NITEM = 10000`: nessun deadlock, nessuna corruzione, CPU non al 100%.
5. Verificate con `valgrind -tool=helgrind`: nessun warning.

Domande di approfondimento

1. Cosa succederebbe se sostituiste il `while` con un `if` prima di `pthread_cond_wait`? In quale scenario si manifesterebbe il bug?
2. Cosa succederebbe se chiamaste `pthread_cond_signal` *prima* di `pthread_mutex_unlock`? E se lo chiamaste *dopo*?
3. Aggiungete un secondo produttore. Il codice funziona ancora correttamente? Perché il `while` è essenziale in questo caso?
4. Misurate il consumo CPU di `step2.c` (busy-wait) e di `step4.c` (CV) con `top` o `htop` mentre girano. Quale differenza osservate?

Soluzione di riferimento

Listing 5: Produttore corretto con condition variable

```

1  void *produttore(void *arg) {
2      Shared *s = (Shared *)arg;
3      for (int i = 0; i < NITEM; i++) {
4          pthread_mutex_lock(&s->mtx);
5
6          /* Aspetta finche' c'e' spazio.
7           * wait rilascia il mutex e sospende atomicamente:
8           * nessuna finestra di race possibile.
9           * Il while ricontrolla al risveglio: necessario
10          * per spurious wakeup e per piu' produttori. */
11          while (s->b.count == N)
12              pthread_cond_wait(&s->not_full, &s->mtx);
13
14          buf_put(&s->b, i);
15
16          /* Segnala al consumatore che c'e' almeno un elemento.
17           * signal basta: un solo slot e' diventato occupato. */

```

```

18     pthread_cond_signal(&s->not_empty);
19     pthread_mutex_unlock(&s->mtx);
20 }
21 return NULL;
22 }

```

Listing 6: Consumatore corretto con condition variable

```

1 void *consumatore(void *arg) {
2     Shared *s = (Shared *)arg;
3     for (int i = 0; i < NITEM; i++) {
4         pthread_mutex_lock(&s->mtx);
5
6         /* Aspetta finche' c'e' almeno un elemento. */
7         while (s->b.count == 0)
8             pthread_cond_wait(&s->not_empty, &s->mtx);
9
10        int v = buf_get(&s->b);
11
12        /* Segnala al produttore che si e' liberato uno slot. */
13        pthread_cond_signal(&s->not_full);
14        pthread_mutex_unlock(&s->mtx);
15
16        printf("Prelevo: %d\n", v);
17    }
18    return NULL;
19 }

```

Riepilogo: cosa abbiamo imparato

step	Problema	Causa
2 — nessuna sincronizzazione	Corruzione dei dati	<code>count++/count-</code> non atomici
3 — solo mutex	Deadlock	Mutex tenuto durante l'attesa
3 — mutex con rilascio	Lost wakeup	Finestra tra <code>unlock</code> e sospensione
4 — mutex + CV	Nessuno	<code>wait</code> è atomica per costruzione

Il percorso mostra che ogni soluzione parziale introduce un problema diverso. Le condition variables non sono una scelta stilistica: sono la primitiva minima necessaria per risolvere tutti e tre i problemi contemporaneamente.